

Unit V: Access Control

Contents:

- 5.1 Access Control Principles
- 5.2 Subjects, Objects and Access Rights
- 5.3 Access Control Matrix and Compatibility Lists
- 5.4 Discretionary Access Control
- 5.5 Role Based Access Control
- 5.6 Attribute Based Access Control
- 5.7 Identity, Credential and Access Management
- 5.8 Trust Frameworks

5.1 Access Control Principles

NISTIR 7298 (Glossary of Key Information Security Terms, May 2013), defines access control as the process of granting or denying specific requests to:

1. obtain and use information and related information processing services; and
2. enter specific physical facilities.

RFC 4949, Internet Security Glossary, defines access control as a process by which use of system resources is regulated according to a security policy and is permitted only by authorized entities (users, programs, processes, or other systems) according to that policy.

We can view access control as a central element of computer security. The principal objectives of computer security are to prevent unauthorized users from gaining access to resources, to prevent legitimate users from accessing resources in an unauthorized manner, and to enable legitimate users to access resources in an authorized manner.

Table below, from NIST SP 800-171 (Protecting Controlled Unclassified Information in Nonfederal Information Systems and Organizations, August 2016), provides a useful list of security requirements for access control services.

Basic Security Requirement
1. Limit information access to authorized users, processes acting on behalf of authorized users, or devices (including other information systems).
2. Limit information system access to the types of transactions and functions that authorized users are permitted to execute.
Derived Security Requirement
3. Control the flow of CUI (Controlled Unclassified Information) in accordance with approved authorizations.
4. Separate the duties of individuals to reduce the risk of malicious activity without consent.
5. Employ the principle of least privilege, including for specific security functions and privileged accounts.
6. Use non-privileged accounts or roles when accessing non-security functions.
7. Prevent non-privileged users from executing privileged functions and audit the execution of such functions.
8. Limit unsuccessful logon attempts.
9. Provide privacy and security notices consistent with applicable CUI rules.

10. Use session lock with pattern-hiding displays to prevent access and viewing of data after period of inactivity.
11. Terminate (automatically) a user session after a defined condition.
12. Monitor and control remote access sessions.
13. Employ cryptographic mechanisms to protect the confidentiality of remote access sessions.
14. Route remote access via managed access control points.
15. Authorize remote execution of privileged commands and remote access to security-relevant information.
16. Authorize wireless access prior to allowing such connections.
17. Protect wireless access using authentication and encryption.
18. Control connection of mobile devices.
19. Encrypt CUI on mobile devices.
20. Verify and control/limit connections to and use of external information systems.
21. Limit use of organizational portable storage devices on external information systems.
22. Control CUI posted or processed on publicly accessible information systems.

Access control implements a security policy that specifies who or what (e.g., in the case of a process) may have access to each specific system resource, and the type of access that is permitted in each instance.

Access Control Context

Figure 4.1 shows a broader context of access control. In addition to access control, this context involves the following entities and functions:

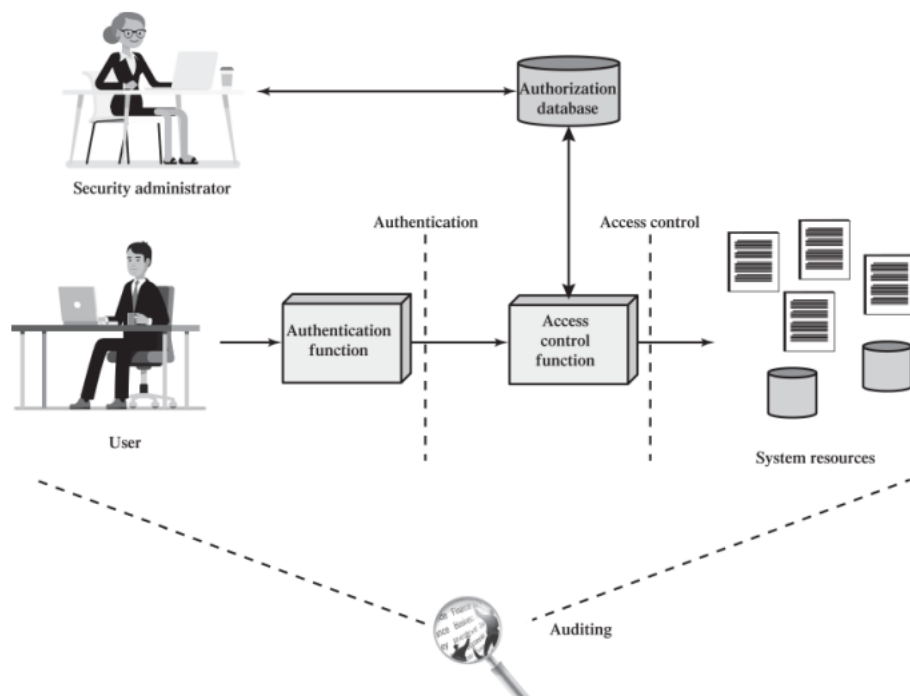


Figure 4.1 Relationship Among Access Control and Other Security Functions

Authentication: Verification that the credentials of a user or other system entity are valid.

Authorization: The granting of a right or permission to a system entity to access a system resource. This function determines who is trusted for a given purpose.

Audit: An independent review and examination of system records and activities in order to test for adequacy of system controls, to ensure compliance with established policy and operational procedures, to detect breaches in security, and to recommend any indicated changes in control, policy, and procedures.

An access control mechanism mediates between a user (or a process executing on behalf of a user) and system resources, such as applications, operating systems, firewalls, routers, files, and databases. The system must first authenticate an entity seeking access. Typically, the authentication function determines whether the user is permitted to access the system at all. Then the access control function determines if the specific requested access by this user is permitted. A security administrator maintains an authorization database that specifies what type of access to which resources is allowed for this user. The access control function consults this database to determine whether to grant access. An auditing function monitors and keeps a record of user accesses to system resources.

Access Control Policies

An access control policy, which can be embodied in an authorization database, dictates what types of access are permitted, under what circumstances, and by whom. Access control policies are generally grouped into the following categories:

- **Discretionary access control (DAC):** Controls access based on the identity of the requestor and on access rules (authorizations) stating what requestors are (or are not) allowed to do. This policy is termed discretionary because an entity might have access rights that permit the entity, by its own decision, to enable another entity to access some resource.
- **Mandatory access control (MAC):** Controls access based on comparing security labels (which indicate how sensitive or critical system resources are) with security clearances (which indicate system entities are eligible to access certain resources). This policy is termed mandatory because an entity that has clearance to access a resource may not, just by its own decision, enable another entity to access that resource.
- **Role-based access control (RBAC):** Controls access based on the roles that users have within the system and on rules stating what accesses are allowed to users in given roles.
- **Attribute-based access control (ABAC):** Controls access based on attributes of the user, the resource to be accessed, and current environmental conditions.

5.2 Subjects, Objects and Access Rights

The basic elements of access control are: subject, object, and access right.

Subject

A subject is an entity capable of accessing objects. Generally, the concept of subject equates with that of process. Any user or application actually gains access to an object by means of a process that represents that user or application. The process takes on the attributes of the user, such as access rights.

A subject is typically held accountable for the actions they have initiated, and an audit trail may be used to record the association of a subject with security-relevant actions performed on an object by the subject. Basic access control systems typically define three classes of subject, with different access rights for each class:

- **Owner:** This may be the creator of a resource, such as a file. For system resources, ownership may belong to a system administrator. For project resources, a project administrator or leader may be assigned ownership.

- **Group:** In addition to the privileges assigned to an owner, a named group of users may also be granted access rights, such that membership in the group is sufficient to exercise these access rights. In most schemes, a user may belong to multiple groups.
- **World:** The least amount of access is granted to users who are able to access the system but are not included in the categories owner and group for this resource.

Object

An object is a resource to which access is controlled. In general, an object is an entity used to contain and/or receive information. Examples include records, blocks, pages, segments, files, portions of files, directories, directory trees, mailboxes, messages, and programs. Some access control systems also encompass, bits, bytes, words, processors, communication ports, clocks, and network nodes.

The number and types of objects to be protected by an access control system depends on the environment in which access control operates and the desired tradeoff between security on the one hand, and complexity, processing burden, and ease of use on the other hand.

Access Right

An access right describes the way in which a subject may access an object. Access rights could include the following:

- **Read:** User may view information in a system resource (e.g., a file, selected records in a file, selected fields within a record, or some combination). Read access includes the ability to copy or print.
- **Write:** User may add, modify, or delete data in system resource (e.g., files, records, programs). Write access includes read access.
- **Execute:** User may execute specified programs.
- **Delete:** User may delete certain system resources, such as files or records. **Create:** User may create new files, records, or fields.
- **Search:** User may list the files in a directory or otherwise search the directory.

5.3 Access Control Matrix and Compatibility Lists

Access Control Matrix

The access control matrix concept was formulated by Lampson, and subsequently refined by Graham and Denning and by Harrison et al.

One dimension of the matrix consists of identified subjects that may attempt data access to the resources. Typically, this list will consist of individual users or user groups, although access could be controlled for terminals, network equipment, hosts, or applications instead of or in addition to users.

The other dimension lists the objects that may be accessed. At the greatest level of detail, objects may be individual data fields. More aggregate groupings, such as records, files, or even the entire database, may also be objects in the matrix.

Each entry in the matrix indicates the access rights of a particular subject for a particular object.

		OBJECTS			
		File 1	File 2	File 3	File 4
SUBJECTS	User A	Own Read Write		Own Read Write	
	User B	Read	Own Read Write	Write	Read
	User C	Read Write	Read		Own Read Write

(a) Access matrix

The figure above, is a simple example of an access matrix. Thus, user A owns files 1 and 3 and has read and write access rights to those files. User B has read access rights to file 1, and so on.

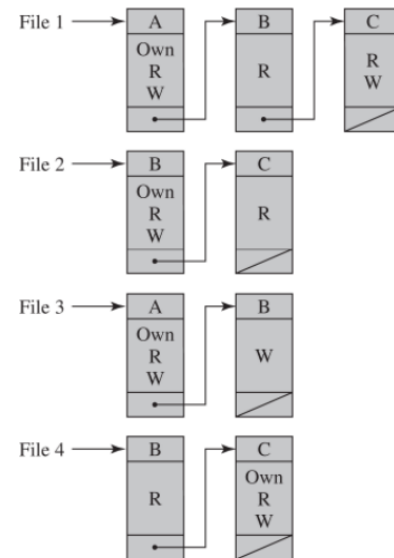
Compatibility Lists

In practice, an access matrix is usually sparse and is implemented by decomposition in one of two ways.

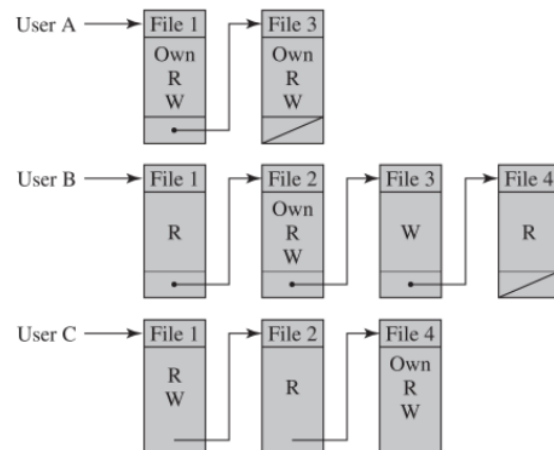
- The matrix may be decomposed by columns, yielding access control lists (ACLs). For each object, an ACL lists users and their permitted access rights. The ACL may contain a default, or public, entry. This allows users that are not explicitly listed as having special rights to have a default set of rights. The default set of rights should always follow the rule of least privilege or read-only access, whichever is applicable. Elements of the list may include individual users as well as groups of users.

When it is desired to determine which subjects have which access rights to a particular resource, ACLs are convenient, because each ACL provides the information for a given resource. However, this data structure is not convenient for determining the access rights available to a specific user.

- Decomposition by rows yields capability tickets (see Figure c). A capability ticket specifies authorized objects and operations for a particular user. Each user has a number of tickets and may be authorized to loan or give them to others. Because tickets may be dispersed around the system, they present a greater security problem than access control lists. The integrity of the ticket must be protected, and guaranteed (usually by the operating system). In particular, the ticket must be unforgeable. One way to accomplish this is to have the operating system hold all tickets on behalf of users. These tickets would have



(b) Access control lists for files of part (a)



(c) Capability lists for files of part (a)

this is to have the operating system hold all tickets on behalf of users. These tickets would have

to be held in a region of memory inaccessible to users. Another alternative is to include an unforgeable token in the capability. This could be a large random password, or a cryptographic message authentication code. This value is verified by the relevant resource whenever access is requested. This form of capability ticket is appropriate for use in a distributed environment, when the security of its contents cannot be guaranteed.

The convenient and inconvenient aspects of capability tickets are the opposite of those for ACLs. It is easy to determine the set of access rights that a given user has, but more difficult to determine the list of users with specific access rights for a specific resource.

5.4 Discretionary Access Control

A discretionary access control scheme is one in which an entity may be granted access rights that permit the entity, by its own decision, to enable another entity to access some resource. A general approach to DAC, as exercised by an operating system or a database management system, is that of an access matrix.

An Access Control Model

It is a general model for DAC developed by Lampson, Graham, and Denning. The model assumes a set of subjects, a set of objects, and a set of rules that govern the access of subjects to objects. Let us define the protection state of a system to be the set of information, at a given point in time that specifies the access rights for each subject with respect to each object. We can identify three requirements:

- representing the protection state,
- enforcing access rights, and
- allowing subjects to alter the protection state in certain ways.

The model addresses all three requirements, giving a general, logical description of a DAC system.

To represent the protection state, we extend the universe of objects in the access control matrix to include the following:

- **Processes:** Access rights include the ability to delete a process, stop (block), and wake up a process.
- **Devices:** Access rights include the ability to read/write the device, to control its operation (e.g., a disk seek), and to block/unblock the device for use.
- **Memory locations or regions:** Access rights include the ability to read/write certain regions of memory that are protected such that the default is to disallow access.
- **Subjects:** Access rights with respect to a subject have to do with the ability to grant or delete access rights of that subject to other objects, as explained subsequently.

		OBJECTS								
		Subjects			Files		Processes		Disk drives	
		S_1	S_2	S_3	F_1	F_2	P_1	P_2	D_1	D_2
SUBJECTS	S_1	control	owner	owner control	read*	read owner	wakeup	wakeup	seek	owner
	S_2		control		write*	execute			owner	seek*
	S_3			control		write	stop			

* = copy flag set

Figure 4.3 Extended Access Control Matrix

Protection Domains

The access control matrix model that we have discussed so far associates a set of capabilities with a user. A more general and more flexible approach is to associate capabilities with protection domains. A protection domain is a set of objects together with access rights to those objects. In terms of the access matrix, a row defines a protection domain. So far, we have equated each row with a specific user. So, in this limited model, each user has a protection domain, and any processes spawned by the user have access rights defined by the same protection domain.

A more general concept of protection domain provides more flexibility. For example, a user can spawn processes with a subset of the access rights of the user, defined as a new protection domain. This limits the capability of the process. Such a scheme could be used by a server process to spawn processes for different classes of users. Also, a user could define a protection domain for a program that is not fully trusted, so its access is limited to a safe subset of the user's access rights.

The association between a process and a domain can be static or dynamic. For example, a process may execute a sequence of procedures and require different access rights for each procedure, such as read file and write file. In general, we would like to minimize the access rights that any user or process has at any one time; the use of protection domains provides a simple means to satisfy this requirement.

One form of protection domain has to do with the distinction made in many operating systems, such as UNIX, between user and kernel mode. A user program executes in a user mode, in which certain areas of memory are protected from the user's use and in which certain instructions may not be executed. When the user process calls a system routine, that routine executes in a system mode, or what has come to be called kernel mode, in which privileged instructions may be executed and in which protected areas of memory may be accessed.

5.5 Role Based Access Control (RBAC)

Traditional DAC systems define the access rights of individual users and groups of users. In contrast, RBAC is based on the roles that users assume in a system rather than the user's identity. Typically, RBAC models define a role as a job function within an organization. RBAC systems assign access rights to roles instead of individual users. In turn, users are assigned to different roles, either statically or dynamically, according to their responsibilities.

RBAC now enjoys widespread commercial use and remains an area of active research. The National Institute of Standards and Technology (NIST) has issued a standard, FIPS PUB 140-3 (Security Requirements for Cryptographic Modules, September 2009), that requires support for access control and administration through roles.

The relationship of users to roles is many to many, as is the relationship of roles to resources, or system objects (see Figure). The set of user's changes, in some environments frequently, and the assignment of a user to one or more roles may also be dynamic. The set of roles in the system in most environments is relatively static, with only occasional additions or deletions. Each role will have specific access rights to one or more resources. The set of resources and the specific access rights associated with a particular role are also likely to change infrequently.

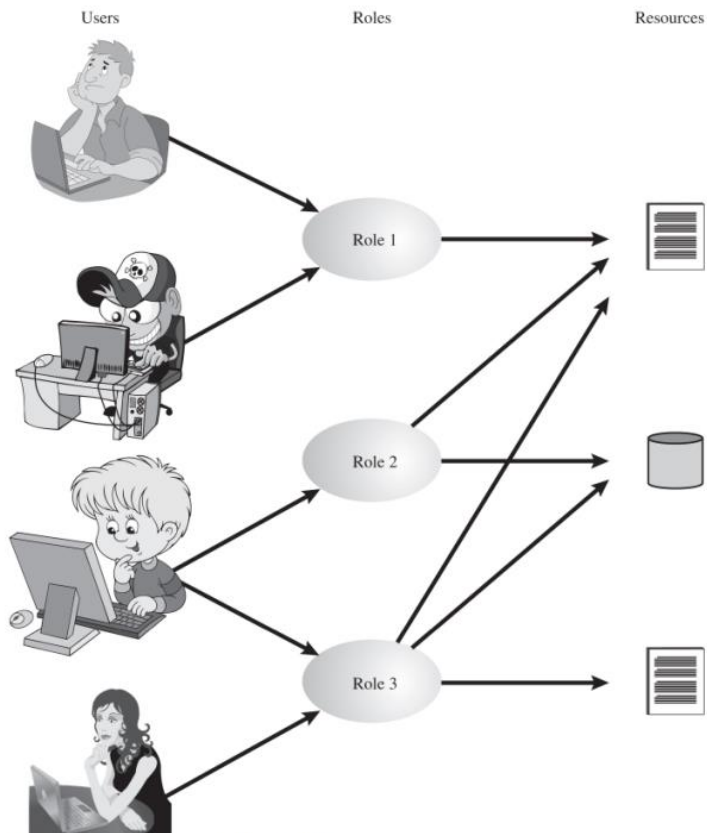


Figure 4.6 Users, Roles, and Resources

We can use the access matrix representation to depict the key elements of an RBAC system in simple terms, as shown in Figure below. The upper matrix relates individual users to roles. Typically there are many more users than roles. Each matrix entry is either blank or marked, the latter indicating that this user is assigned to this role. Note a single user may be assigned multiple roles (more than one mark in a row) and multiple users may be assigned to a single role (more than one mark in a column). The lower matrix has the same structure as the DAC access control matrix, with roles as subjects. Typically, there are few roles and many objects, or resources. In this matrix, the entries are the specific access rights enjoyed by the roles. Note a role can be treated as an object, allowing the definition of role hierarchies.

	R_1	R_2	$\dots$	R_n
U_1	×			
U_2	×			
U_3		×		×
U_4				×
U_5				×
U_6				×
$\vdots$				
U_m	×			

		OBJECTS								
		R ₁	R ₂	R _n	F ₁	F ₂	P ₁	P ₂	D ₁	D ₂
ROLES	R ₁	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
	R ₂		control		write *	execute			owner	seek *
	•									
	•									
	•									
	R _n			control		write	stop			

Figure 4.7 Access Control Matrix Representation of RBAC

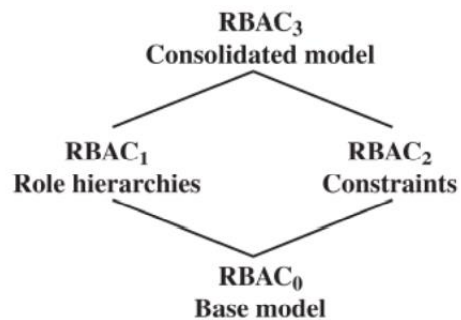
RBAC lends itself to an effective implementation of the principle of least privilege. Each role should contain the minimum set of access rights needed for that role. A user is assigned to a role that enables him or her to perform only what is required for that role. Multiple users assigned to the same role enjoy the same minimal set of access rights.

RBAC Reference Models

A variety of functions and services can be included under the general RBAC approach. To clarify the various aspects of RBAC, it is useful to define a set of abstract models of RBAC functionality.

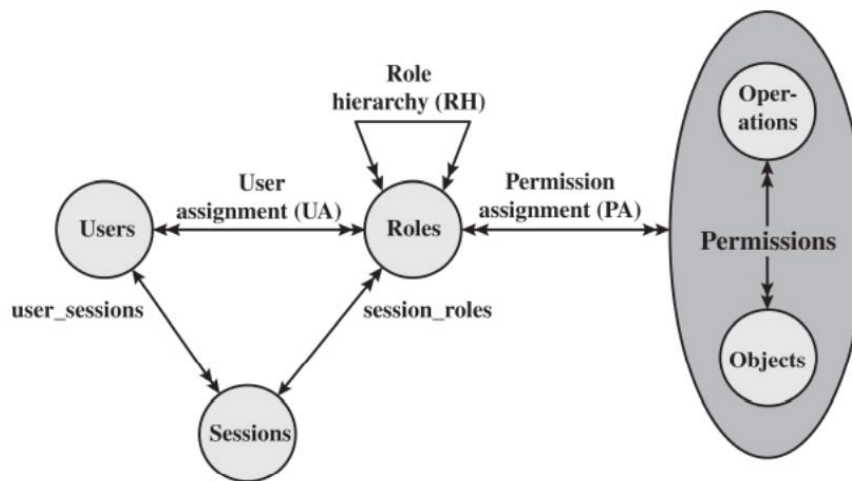
There is a family of reference models that has served as the basis for ongoing standardization efforts. This family consists of four models that are related to each other, as shown in Figure.

- RBAC0 contains the minimum functionality for an RBAC system.
- RBAC1 includes the RBAC0 functionality and adds role hierarchies, which enable one role to inherit permissions from another role.
- RBAC2 includes RBAC0 and adds constraints, which restrict the ways in which the components of an RBAC system may be configured.
- RBAC3 contains the functionality of RBAC0, RBAC1 and RBAC2.



(a) Relationship among RBAC models

Models	Hierarchies	Constraints
RBAC0	No	No
RBAC1	Yes	No
RBAC2	No	Yes
RBAC3	Yes	Yes



Base MODEL—RBAC0

RBAC0 system contains the four types of entities without the role hierarchy and constrains.

- **User:** An individual that has access to this computer system. Each individual has an associated user ID.
- **Role:** A named job function within the organization that controls this computer system. Typically, associated with each role is a description of the authority and responsibility conferred on this role, and on any user who assumes this role.
- **Permission:** An approval of a particular mode of access to one or more objects. Equivalent terms are access right, privilege, and authorization.
- **Session:** A mapping between a user and an activated subset of the set of roles to which the user is assigned.

The arrowed lines in Figure above indicate relationships, or mappings, with a single arrowhead indicating one, and a double arrowhead indicating many. Thus, there is a many-to-many relationship between users and roles: One user may have multiple roles, and multiple users may be assigned to a single role. Similarly, there is a many-to-many relationship between roles and permissions. A session is used to define a temporary one-to-many relationship between a user and one or more of the roles to which the user has been assigned. The user establishes a session with only the roles needed for a particular task; this is an example of the concept of least privilege.

The many-to-many relationships between users and roles and between roles and permissions provide a flexibility and granularity of assignment not found in conventional DAC schemes. Without this flexibility and granularity, there is a greater risk that a user may be granted more access to resources than is needed because of the limited control over the types of access that can be allowed. The NIST RBAC document gives the following examples: Users may need to list directories and modify existing files without creating new files, or they may need to append records to a file without modifying existing records.

Role Hierarchies – RBAC1

Role hierarchies provide a means of reflecting the hierarchical structure of roles in an organization. Typically, job functions with greater responsibility have greater authority to access resources. A subordinate job function may have a subset of the access rights of the superior job function. Role hierarchies make use of the concept of inheritance to enable one role to implicitly include access rights associated with a subordinate role.

Figure 4.9 is an example of a diagram of a role hierarchy. By convention, subordinate roles are lower in the diagram. A line between two roles implies the upper role includes all of the access rights of the lower role, as well as other access rights not available to the lower role. One role can inherit access rights from multiple subordinate roles. For example, in Figure 4.9, the Project Lead role includes all of the access rights of the Production Engineer role and of the Quality Engineer role. More than one role can inherit from the same subordinate role. For example, both the Production Engineer role and the Quality Engineer role include all of the access rights of the Engineer role. Additional access rights are also assigned to the Production Engineer Role, and a different set of additional access rights are assigned to the Quality Engineer role. Thus, these two roles have overlapping access rights, namely, the access rights they share with the Engineer role.

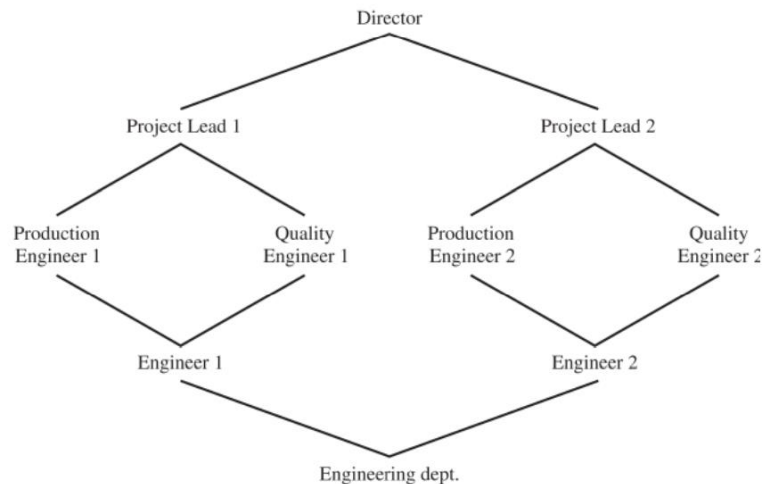


Figure 4.9 Example of Role Hierarchy

Constraints – RBAC2

Constraints provide a means of adapting RBAC to the specifics of administrative and security policies in an organization. A constraint is a defined relationship among roles or a condition related to roles. Different types of constraints: mutually exclusive roles, cardinality, and prerequisite roles.

Mutually exclusive roles are roles such that a user can be assigned to only one role in the set. This limitation could be a static one, or it could be dynamic, in the sense that a user could be assigned only one of the roles in the set for a session. The mutually exclusive constraint supports a separation of duties and capabilities within an organization. This separation can be reinforced or enhanced by use of mutually exclusive permission assignments. With this additional constraint, a mutually exclusive set of roles has the following properties:

1. A user can only be assigned to one role in the set (either during a session or statically).
2. Any permission (access right) can be granted to only one role in the set.

Thus, the set of mutually exclusive roles have non overlapping permissions. If two users are assigned to different roles in the set, then the users have non overlapping permissions while assuming those roles. The purpose of mutually exclusive roles is to increase the difficulty of collusion among individuals of different skills or divergent job functions to thwart security policies.

Cardinality refers to setting a maximum number with respect to roles. One such constraint is to set a maximum number of users that can be assigned to a given role. For example, a project leader role or a department head role might be limited to a single user. The system could also impose a constraint on the number of roles that a user is assigned to, or the number of roles a user can activate for a single session. Another form of constraint is to set a maximum number of roles that can be granted a particular permission; this might be a desirable risk mitigation technique for a sensitive or powerful permission.

A system might be able to specify a **prerequisite role**, which dictates a user can only be assigned to a particular role if it is already assigned to some other specified role. A prerequisite can be used to structure the implementation of the least privilege concept. In a hierarchy, it might be required that a user can be assigned to a senior (higher) role only if it is already assigned an immediately junior (lower) role. For example, in Figure 4.9 a user assigned to a Project Lead role must also be assigned to the subordinate Production Engineer and Quality Engineer roles. Then, if the user does not need all of the permissions of the Project Lead role for a given task, the user can invoke a session using only the required subordinate role. Note the use of prerequisites tied to the concept of hierarchy requires the RBAC3 model.

5.6 Attribute Based Access Control (ABAC)

An ABAC model can define authorizations that express conditions on properties of both the resource and the subject. For example, consider a configuration in which each resource has an attribute that identifies the subject that created the resource. Then, a single access rule can specify the ownership privilege for all the creators of every resource. The strength of the ABAC approach is its flexibility and expressive power. Its main obstacle to its adoption in real systems has been concern about the performance impact of evaluating predicates on both resource and user properties for each access. However, for applications such as cooperating Web services and cloud computing, this increased performance cost is less noticeable because there is already a relatively high performance cost for each access. Thus, Web services have been pioneering technologies for implementing ABAC models, especially through the introduction of the eXtensible Access Control Markup Language (XAMCL), and there is considerable interest in applying the ABAC model to cloud services.

There are three key elements to an ABAC model:

- attributes, which are defined for entities in a configuration;
- a policy model, which defines the ABAC policies; and
- the architecture model, which applies to policies that enforce access control.

Attributes

Attributes are characteristics that define specific aspects of the subject, object, environment conditions, and/or requested operations that are predefined and preassigned by an authority. Attributes contain information that indicates the class of information given by the attribute, a name, and a value.

Example: Class=HospitalRecordsAccess, Name=PatientInformationAccess, Value=MFBusinessHoursOnly)

The following are the three types of attributes in the ABAC model:

- **Subject attributes:** A subject is an active entity (e.g., a user, an application, a process, or a device) that causes information to flow among objects or changes the system state. Each subject has associated attributes that define the identity and characteristics of the subject. Such attributes may include the subject's identifier, name, organization, job title, and so on. A subject's role can also be viewed as an attribute.
- **Object attributes:** An object, also referred to as a resource, is a passive (in the context of the given request) information system-related entity (e.g., devices, files, records, tables, processes, programs, networks, domains) containing or receiving information. As with subjects, objects have attributes that can be leveraged to make access control decisions. A Microsoft Word document, for example, may have attributes such as title, subject, date, and author. Object attributes can often be extracted from the metadata of the object. In particular, a variety of Web service

metadata attributes may be relevant for access control purposes, such as ownership, service taxonomy, or even Quality of Service (QoS) attributes.

- **Environment attributes:** These attributes have so far been largely ignored in most access control policies. They describe the operational, technical, and even situational environment or context in which the information access occurs. For example, attributes, such as current date and time, the current virus/hacker activities, and the network's security level (e.g., Internet vs. intranet), are not associated with a particular subject nor a resource, but may nonetheless be relevant in applying an access control policy.

ABAC is a logical access control model that is distinguishable because it controls access to objects by evaluating rules against the attributes of entities (subject and object), operations, and the environment relevant to a request. ABAC relies upon the evaluation of attributes of the subject, attributes of the object, and a formal relationship or access control rule defining the allowable operations for subject-object attribute combinations in a given environment. All ABAC solutions contain these basic core capabilities to evaluate attributes and enforce rules or relationships between those attributes. ABAC systems are capable of enforcing DAC, RBAC, and MAC concepts. ABAC enables fine-grained access control, which allows for a higher number of discrete inputs into an access control decision, providing a bigger set of possible combinations of those variables to reflect a larger and more definitive set of possible rules, policies, or restrictions on access. Thus, ABAC allows an unlimited number of attributes to be combined to satisfy any access control rule. Moreover, ABAC systems can be implemented to satisfy a wide array of requirements from basic access control lists through advanced expressive policy models that fully leverage the flexibility of ABAC.

ABAC Logical Architecture

Figure 4.10 illustrates in a logical architecture the essential components of an ABAC system. An access by a subject to an object proceeds according to the following steps:

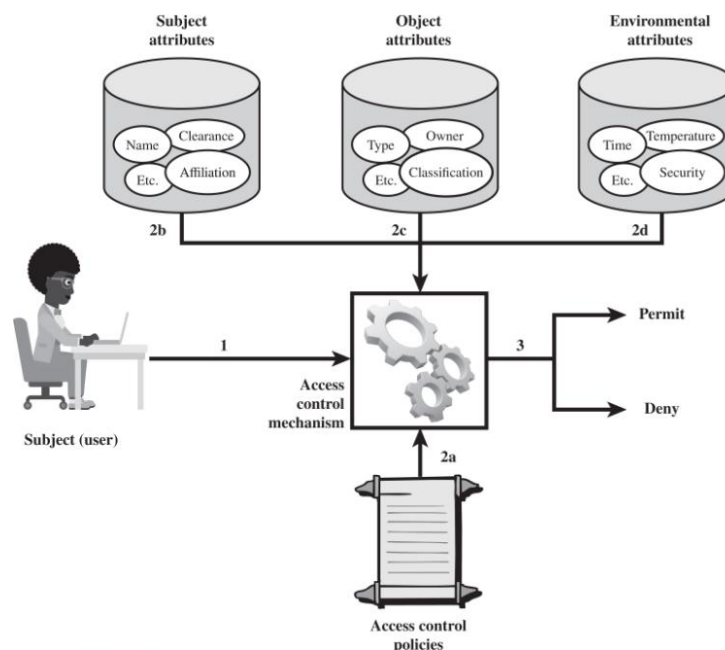


Figure 4.10 ABAC Scenario

1. A subject requests access to an object. This request is routed to an access control mechanism.
2. The access control mechanism is governed by a set of rules (2a) that are defined by a preconfigured access control policy. Based on these rules, the access control mechanism assesses

the attributes of the subject (2b), object (2c), and current environmental conditions (2d) to determine authorization.

3. The access control mechanism grants the subject access to the object if access is authorized, and denies access if it is not authorized.

It is clear from the logical architecture that there are four independent sources of information used for the access control decision. The system designer can decide which attributes are important for access control with respect to subjects, objects, and environmental conditions. The system designer or other authority can then define access control policies, in the form of rules, for any desired combination of attributes of subject, object, and environmental conditions. It should be evident that this approach is very powerful and flexible. However, the cost, both in terms of the complexity of the design and implementation, and in terms of the performance impact, is likely to exceed that of other access control approaches. This is a trade-off that the system authority must make.

ABAC Policies

A policy is a set of rules and relationships that govern allowable behavior within an organization, based on the privileges of subjects and how resources or objects are to be protected under which environment conditions. In turn, privileges represent the authorized behavior of a subject; they are defined by an authority and embodied in a policy. Other terms that are commonly used instead of privileges are rights, authorizations, and entitlements. Policy is typically written from the perspective of the object that needs protecting, and the privileges available to subjects.

5.7 Identity, Credential and Access Management (ICAM)

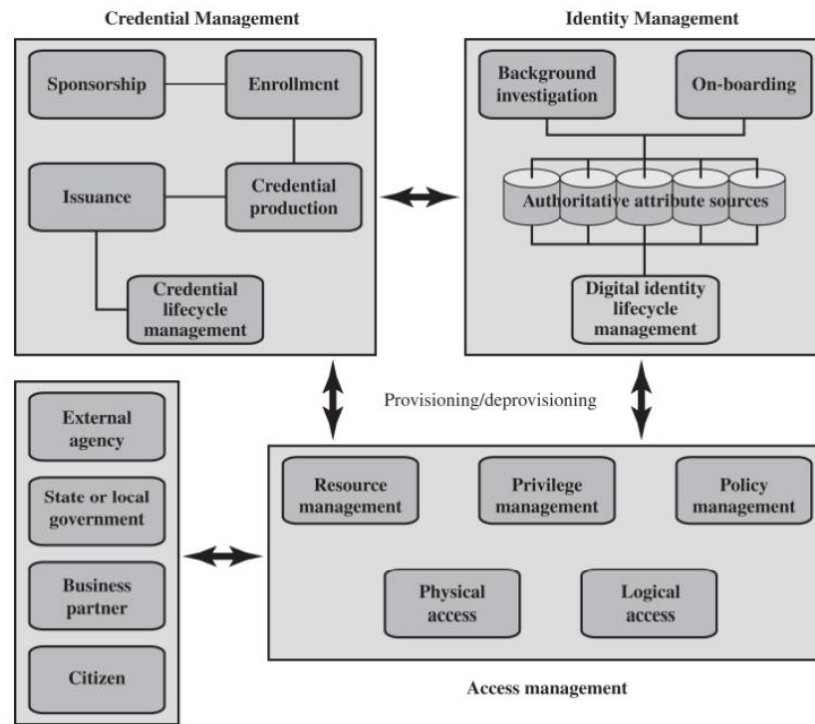
ICAM is a comprehensive approach to managing and implementing digital identities (and associated attributes), credentials, and access control. ICAM has been developed by the U.S. government, but is applicable not only to government agencies, but also may be deployed by enterprises looking for a unified approach to access control. ICAM is designed to:

- Create trusted digital identity representations of individuals and what the ICAM documents refer to as nonperson entities (NPEs). The latter include processes, applications, and automated devices seeking access to a resource.
- Bind those identities to credentials that may serve as a proxy for the individual or NPE in access transactions. A credential is an object or data structure that authoritatively binds an identity (and optionally, additional attributes) to a token possessed and controlled by a subscriber.
- Use the credentials to provide authorized access to an agency's resources.

Figure 4.12 provides an overview of the logical components of an ICAM architecture. We will examine each of the main components in the following subsections.

Identity Management

Identity management is concerned with assigning attributes to a digital identity and connecting that digital identity to an individual or NPE. The goal is to establish a trustworthy digital identity that is independent of a specific application or context. The traditional, and still most common, approach to access control for applications and programs is to create a digital representation of an identity for the specific use of the application or program. As a result, maintenance and protection of the identity itself is treated as secondary to the mission associated with the application. Further, there is considerable overlap in effort in establishing these application specific identities.



Identity federation
Figure 4.12 Identity, Credential, and Access Management (ICAM)

Unlike accounts used to log on to networks, systems, or applications, enterprise identity records are not tied to job title, job duties, location, or whether access is needed to a specific system. Those items may become attributes tied to an enterprise identity record, and may also become part of what uniquely identifies an individual in a specific application. Access control decisions will be based on the context and relevant attributes of a user—not solely their identity. The concept of an enterprise identity is that individuals will have a single digital representation of themselves that can be leveraged across departments and agencies for multiple purposes, including access control.

Figure 4.12 depicts the key functions involved in identity management. Establishment of a digital identity typically begins with collecting identity data as part of an enrollment process. A digital identity is often comprised of a set of attributes that when aggregated uniquely identify a user within a system or an enterprise. In order to establish trust in the individual represented by a digital identity, an agency may also conduct a background investigation. Attributes about an individual may be stored in various authoritative sources within an agency and linked to form an enterprise view of the digital identity. This digital identity may then be provisioned into applications in order to support physical and logical access (part of Access Management) and de-provisioned when access is no longer required.

A final element of identity management is lifecycle management, which includes the following:

- Mechanisms, policies, and procedures for protecting personal identity information
- Controlling access to identity data
- Techniques for sharing authoritative identity data with applications that need it
- Revocation of an enterprise identity

Credential Management

As mentioned, a credential is an object or data structure that authoritatively binds an identity (and optionally, additional attributes) to a token possessed and controlled by a subscriber. Examples of

credentials are smart cards, private/public cryptographic keys, and digital certificates. Credential management is the management of the life cycle of the credential. Credential management encompasses the following five logical components:

1. An authorized individual sponsors an individual or entity for a credential to establish the need for the credential. For example, a department supervisor sponsors a department employee.
2. The sponsored individual enrolls for the credential, a process which typically consists of identity proofing and the capture of biographic and biometric data. This step may also involve incorporating authoritative attribute data, maintained by the identity management component.
3. A credential is produced. Depending on the credential type, production may involve encryption, the use of a digital signature, the production of a smartcard, or other functions.
4. The credential is issued to the individual or NPE.
5. Finally, a credential must be maintained over its life cycle, which might include revocation, reissuance/replacement, reenrollment, expiration, personal identification number (PIN) reset, suspension, or reinstatement.

Access Management

The access management component deals with the management and control of the ways entities are granted access to resources. It covers both logical and physical access, and may be internal to a system or an external element. The purpose of access management is to ensure that the proper identity verification is made when an individual attempts to access security-sensitive buildings, computer systems, or data. The access control function makes use of credentials presented by those requesting access and the digital identity of the requestor. Three support elements are needed for an enterprise-wide access control facility:

- Resource management: This element is concerned with defining rules for a resource that requires access control. The rules would include credential requirements and what user attributes, resource attributes, and environmental conditions are required for access of a given resource for a given function.
- Privilege management: This element is concerned with establishing and maintaining the entitlement or privilege attributes that comprise an individual's access profile. These attributes represent features of an individual that can be used as the basis for determining access decisions to both physical and logical resources. Privileges are considered attributes that can be linked to a digital identity.
- Policy management: This element governs what is allowable and unallowable in an access transaction. That is, given the identity and attributes of the requestor, the attributes of the resource or object, and environmental conditions, a policy specifies what actions this user can perform on this object.

Identity Federation

Identity federation addresses two questions:

1. How do you trust identities of individuals from external organizations who need access to your systems?
2. How do you vouch for identities of individuals in your organization when they need to collaborate with external organizations?

Identity federation is a term used to describe the technology, standards, policies, and processes that allow an organization to trust digital identities, identity attributes, and credentials created and issued by another organization.

5.8 Trust Frameworks

The interrelated concepts of trust, identity, and attributes have become core concerns of Internet businesses, network service providers, and large enterprises. These concerns can clearly be seen in the e-commerce setting. For efficiency, privacy, and legal simplicity, parties to transactions generally apply the need-to-know principle: What do you need to know about someone in order to deal with them? The answer varies from case to case, and includes such attributes as professional registration or license number, organization and department, staff ID, security clearance, customer reference number, credit card number, unique health identifier, allergies, blood type, Social Security number, address, citizenship status, social networking handle, pseudonym, and so on. The attributes of an individual that must be known and verified to permit a transaction depend on context.

The same concern for attributes is increasingly important for all types of access control situations, not just the e-business context. For example, an enterprise may need to provide access to resources for customers, users, suppliers, and partners. Depending on context, access will be determined not just by identity, but by the attributes of the requestor and the resource.

Traditional Identity Exchange Approach

Online or network transactions involving parties from different organizations, or between an organization and an individual user such as an online customer, generally require the sharing of identity information. This information may include a host of associated attributes in addition to a simple name or numerical identifier. Both the party disclosing the information and the party receiving the information need to have a level of trust about security and privacy issues related to that information.

Figure 4.13 a shows the traditional technique for the exchange of identity information. This involves users developing arrangements with an identity service provider to procure digital identity and credentials, and arrangements with parties that provide end-user services and applications and that are willing to rely on the identity and credential information generated by the identity service provider.

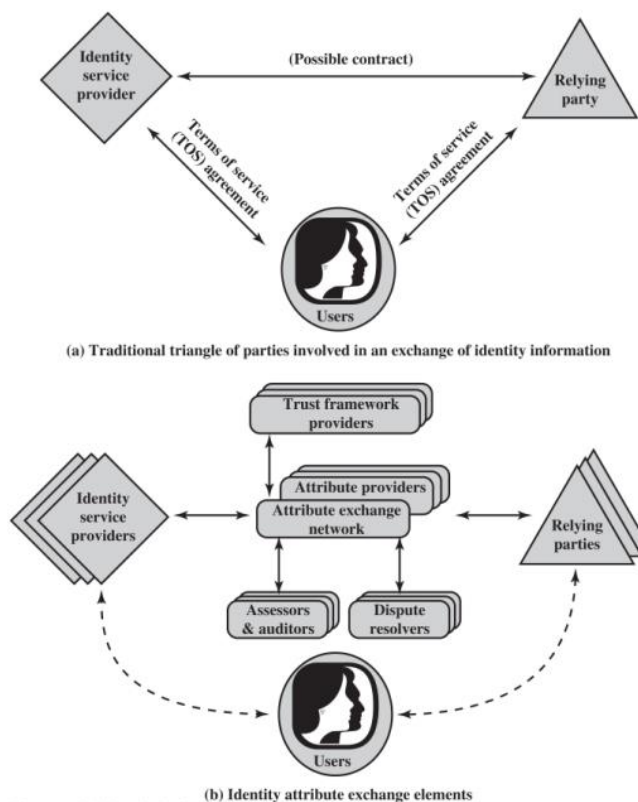


Figure 4.13 Identity Information Exchange Approaches

The arrangement of Figure 4.13a must meet a number of requirements. The relying party requires that the user has been authenticated to some degree of assurance, that the attributes imputed to the user by

the identity service provider are accurate, and that the identity service provider is authoritative for those attributes. The identity service provider requires assurance that it has accurate information about the user and that, if it shares information, the relying party will use it in accordance with contractual terms and conditions and the law. The user requires assurance that the identity service provider and relying party can be entrusted with sensitive information and that they will abide by the user's preferences and respect the user's privacy. Most importantly, all the parties want to know if the practices described by the other parties are actually those implemented by the parties, and how reliable those parties are.

Open Identity Trust Framework

Without some universal standard and framework, the arrangement of Figure 4.13a must be replicated in multiple contexts. A far preferable approach is to develop an open, standardized approach to trustworthy identity and attribute exchange.

Unfortunately, this topic is burdened with numerous acronyms, so it is best to begin with a definition of the most important of these:

- **OpenID:** This is an open standard that allows users to be authenticated by certain cooperating sites (known as Relying Parties) using a third party service, eliminating the need for Webmasters to provide their own ad hoc systems and allowing users to consolidate their digital identities. Users may create accounts with their preferred OpenID identity providers, then use those accounts as the basis for signing on to any Web site that accepts OpenID authentication.
- **OIDF:** The OpenID Foundation is an international nonprofit organization of individuals and companies committed to enabling, promoting, and protecting OpenID technologies. OIDF assists the community by providing needed infrastructure and help in promoting and supporting expanded adoption of OpenID.
- **ICF:** The Information Card Foundation is a nonprofit community of companies and individuals working together to evolve the Information Card ecosystem. Information Cards are personal digital identities people can use online, and the key component of identity metasystems. Visually, each Information Card has a card-shaped picture and a card name associated with it that enable people to organize their digital identities and to easily select one they want to use for any given interaction.
- **OITF:** The Open Identity Trust Framework is a standardized, open specification of a trust framework for identity and attribute exchange, developed jointly by OIDF and ICF.
- **OIX:** The Open Identity Exchange Corporation is an independent, neutral, international provider of certification trust frameworks conforming to the Open Identity Trust Frameworks model.
- **AXN:** An Attribute Exchange Network (AXN) is an online Internet-scale gateway for identity service providers and relying parties to efficiently access user-asserted, permissioned, and verified online identity attributes in high volumes at affordable costs.

System managers need to be able to trust that the attributes associated with a subject or an object are authoritative and are exchanged securely. One approach to providing that trust within an organization is the ICAM model, specifically the ICAM components (see Figure 4.12). Combined with an identity federation functionality that is shared with other organizations, attributes can be exchanged in a trustworthy fashion, supporting secure access control.

In digital identity systems, a trust framework functions as a certification program. It enables a party who accepts a digital identity credential (called the relying party) to trust the identity, security, and privacy policies of the party who issues the credential (called the identity service provider) and vice versa. More

formally, OIX defines a trust framework as a set of verifiable commitments from each of the various parties in a transaction to their counter parties. These commitments include

1. controls (including regulatory and contractual obligations) to help ensure commitments are delivered and
2. remedies for failure to meet such commitments.

A trust framework is developed by a community whose members have similar goals and perspectives. It defines the rights and responsibilities of that community's participants; specifies the policies and standards specific to the community; and defines the community-specific processes and procedures that provide assurance. Different trust frameworks can exist, and sets of participants can tailor trust frameworks to meet their particular needs.

Figure 4.13b shows the elements involved in the OITF. Within any given organization or agency, the following roles are part of the overall framework:

- **Relying parties (RPs):** Also called service providers, these are entities delivering services to specific users. RPs must have confidence in the identities and/or attributes of their intended users, and must rely upon the various credentials presented to evince those attributes and identities.
- **Subjects:** These are users of an RP's services, including customers, employees, trading partners, and subscribers.
- **Attribute providers (APs):** APs are entities acknowledged by the community of interest as being able to verify given attributes as presented by subjects and which are equipped through the AXN to create conformant attribute credentials according to the rules and agreements of the AXN. Some APs will be sources of authority for certain information; more commonly APs will be brokers of derived attributes.
- **Identity providers (IDPs):** These are entities able to authenticate user credentials and to vouch for the names (or pseudonyms or handles) of subjects, and which are equipped through the AXN or some other compatible Identity and Access Management (IDAM) system to create digital identities that may be used to index user attributes.

There are also the following important support elements as part on an AXN:

- **Assessors:** Assessors evaluate identity service providers and RPs and certify that they are capable of following the OITF provider's blueprint.
- **Auditors:** These entities may be called on to check that parties' practices have been in line with what was agreed for the OITF.
- **Dispute resolvers:** These entities provide arbitration and dispute resolution under OIX guidelines.
- **Trust framework providers:** A trust framework provider is an organization that translates the requirements of policymakers into an own blueprint for a trust framework that it then proceeds to build, doing so in a way that is consistent with the minimum requirements set out in the OITF specification. In almost all cases, there will be a reasonably obvious candidate organization to take on this role, for each industry sector or large organization that decides it is appropriate to interoperate with an AXN.

The solid arrowed lines in Figure 4.13b indicate agreements with the trust framework provider for implementing technical, operations, and legal requirements. The dashed arrowed lines indicate other agreements potentially affected by these requirements. In general terms, the model illustrated in Figure 4.13b would operate in the following way. Responsible persons within participating organizations determine the technical, operational, and legal requirements for exchanges of identity information that fall under their authority. They then select OITF providers to implement these requirements. These OITF

providers translate the requirements into a blueprint for a trust framework that may include additional conditions of the OITF provider. The OITF provider vets identity service providers and RPs and contracts with them to follow its trust framework requirements when conducting exchanges of identity information. The contracts carry provisions relating to dispute resolvers, and auditors for contract interpretation and enforcement.

Practice Questions

1. Briefly define the difference between DAC and MAC.
2. How does RBAC relate to DAC and MAC?
3. List and define the three classes of subject in an access control system.
4. In the context of access control, what is the difference between a subject and an object?
5. What is an access right?
6. What is the difference between an access control list and a capability ticket?
7. What is a protection domain?
8. Briefly define the four RBAC models.
9. List and define the four types of entities in a base model RBAC system.
10. Describe three types of role hierarchy constraints.
11. Explain the major elements of ABAC model.
12. What is Identity, Credential and Access Management (ICAM)? Explain in detail.
13. Explain the concept of trust framework.